



Norwegian University of
Science and Technology

DEPARTMENT OF COMPUTER SCIENCE

TDT4200 - PARALLEL PROGRAMMING

Exercise 1

1 Introduction

This exercise will prepare you for the baseline code used for the rest of the assignments this year. You will implement a 1D numerical solution to the Wave Equation using the **explicit** Finite Differences Method (FDM) with **central differences** (meaning you iterate forward in time and approximate derivatives by using values systematically spaced on both sides).

The Wave Equation is given by

$$\frac{\partial^2 u}{\partial t^2} = c^2 \cdot \frac{\partial^2 u}{\partial x^2}, \quad (1)$$

where

- u is the wave's amplitude
- t is time
- x is space
- c is the wave's propagation rate (celerity)

In 1D, Equation 1 can be visualised as movement on a string or a wire, like a jumping rope pulled up and down, like in Fig. 1. In the 2D case, you can think of it as waves moving through the ocean.

$N = 9$



Figure 1: A 1D "wave" discretised with 9 points.

Fig. 2 and Fig. 7 show the initial state when the code is successfully implemented.

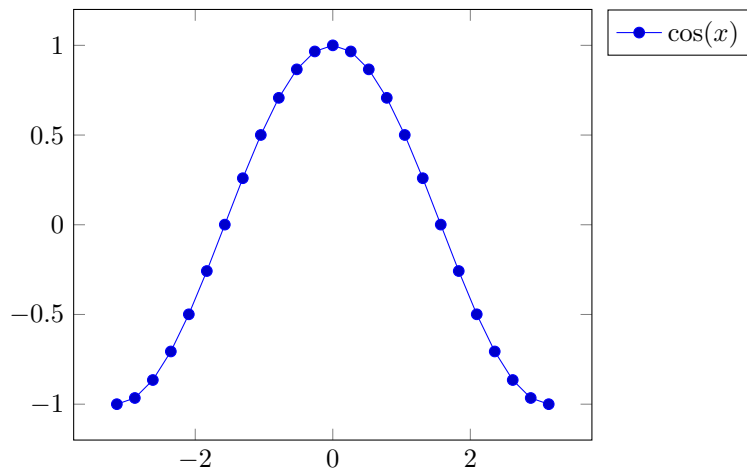


Figure 2: Fig. 1 illustrated on a 2D plane with $\cos 2\pi x$ as the initial condition.

FDM is based on discretising the space and time domains with a finite set of points and approximating the values at the end of the solution by solving algebraic equations using nearby points. Figure 3 illustrates this, where all the rows are in the same space at different times.

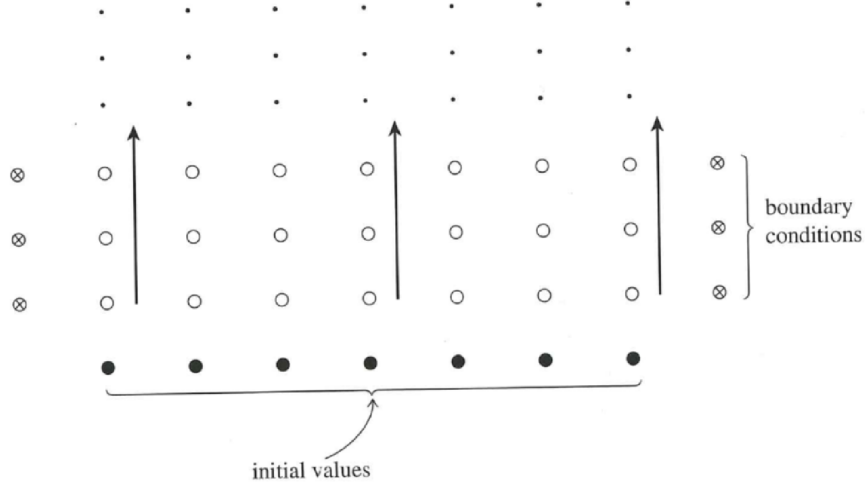


Figure 3: Finite Differences. The dots represent discrete points in space at different time steps. Remember, 1D is a line.

We will be using **central differences**. Let $\Delta x = h$, and substituting the central difference approximation

$$f''(x) \approx \frac{f(x-h) - 2 \cdot f(x) + f(x+h)}{h^2}$$

into Equation 1 for space yields

$$\frac{\partial^2 u}{\partial t^2} = c^2 \cdot \frac{u_{i-1}^t + u_{i+1}^t - 2u_i^t}{h^2}, \quad (2)$$

where u_a^b denotes $u(a \cdot \Delta x, b \cdot \Delta t) = u(x, t)$ for brevity.

Similarly, we substitute

$$f''(t) \approx \frac{f(t-\Delta t) + f(t+\Delta t) - 2 \cdot f(t)}{\Delta t^2}$$

into Equation 2 to obtain

$$\frac{u_i^{t-1} - 2 \cdot u_i^t + u_i^{t+1}}{\Delta t^2} = c^2 \cdot \frac{u_{i-1}^t + u_{i+1}^t - 2u_i^t}{h^2} \quad (3)$$

as the numerical equation we need to solve.

2 Tasks

2.1 Programming

In the wave equation, we need to store three time steps, the current and the previous time iteration, to approximate the next (future) time iteration. All the tasks are marked and can be found in the file `wave_1d.c`. The attached `README.md` explains the `Makefile` and how to run the code. The code is buildable and runnable out-of-the-box, but it will only produce a single, empty `.dat` file in the `data` folder when run. When the code is correctly implemented, the commands `make plot` and `make movie` will function as intended, creating `.png` plots and an `.mp4` movie.

Tasks are structured with a task number, a bit of background text, and the specifics of the task itself in the grey box.

1. Initialise the domain:

The first thing we need to do is to initialise the domain and make it ready for simulation. We apply

$$u(x, 0) = \cos(2\pi x) \quad (4)$$

as the initial condition of the system, where x is a point in space.

The time step in the wave equation is usually derived from the Courant-Friedrichs-Lewy (CFL) condition to ensure stability in the numerical scheme. It is given by

$$\Delta t \leq \frac{\Delta x}{c}. \quad (5)$$

Implement the function `domain_initialize`. The function should:

- Allocate memory for three sets of spatial domains: the previous, the current, and the next domain.
- Apply the initial condition from Equation 4.
- Set the time step, `dt`, based on Equation 5.

2. Finalise the domain:

After allocating memory, we must also return it to the OS to prevent memory leaks.

Implement the function `domain_finalize`. The function should free the memory allocated by `domain_initialize`.

3. Move the buffer

In T1, you allocate memory for three sets of spatial domains in the buffer.

```
1 // Buffers for three time steps, indexed with two ghost points for the boundary.
2 real_t
3     *buffers[3] = { NULL, NULL, NULL };
```

Each time we do a time step, the oldest spatial domain is no longer needed, and the newly calculated spatial domain should be the current one. This is illustrated in Figure 4, where each square represents a space domain at a particular time step.

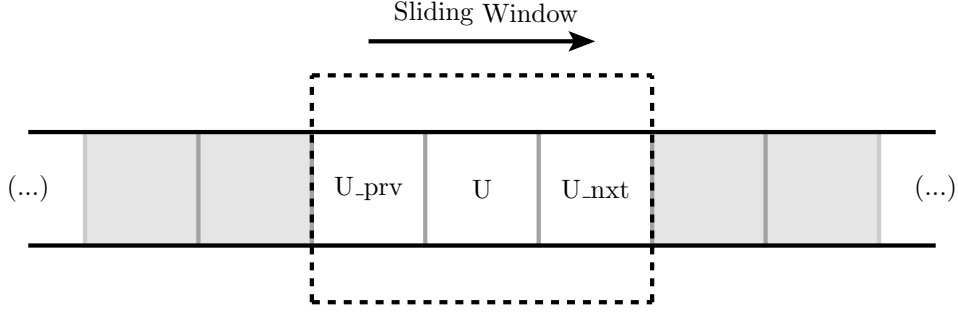


Figure 4: The window moves one time iteration to the right as a new state is calculated.

Define and implement a function that shifts the buffer domain window forward by one step.

4. Time Step

To integrate forward in time, we solve Equation 3 for u_i^{t+1} which yields

$$u_i^{t+1} = -u_i^{t-1} + 2u_i^t + \frac{\Delta t^2 c^2}{h^2} \cdot (u_{i-1}^t + u_{i+1}^t - 2u_i^t) \quad (6)$$

for every point in space. Figure 5 illustrates the process.

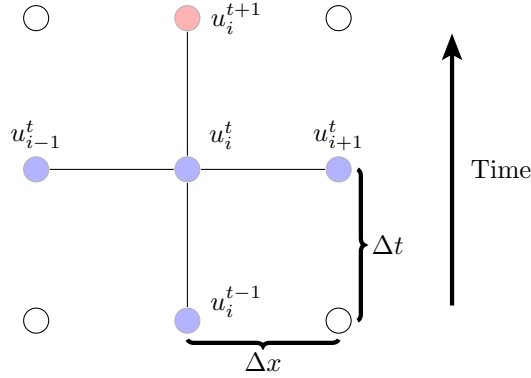


Figure 5: A visualisation of Equation 6. To calculate the new point (red), we use the already-known points (blue).

Define and implement a function that does one iteration of integration over the time domain (does one step forward in time). The function should calculate the next time iteration based on Equation 6 for every point in space.

5. Boundary Conditions

We have added two ghost points to calculate the next point in space, positioned directly adjacent to the boundary. The boundary conditions are used when we are at the edge of our domain. In this solution, we utilise the Neumann (reflective) boundary condition. This means the ghost cell mirrors the point's value one step further into the space domain, as illustrated in Figure 6.

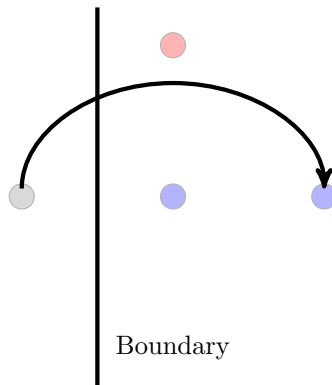


Figure 6: The ghost point outside the boundary (grey) copies the value of the cell one step inside the boundary as a reflection. This point is needed to calculate the next point in time (red).

Remember that there is a boundary on each side of the domain (left and right).

Define and implement a function that sets the value of the ghost cell to mirror that of the point one further into the domain.

6. Simulate

Now, it's time to combine the whole solution. Additionally, we want to observe how the wave progresses. Therefore, we want to save the state at the frequency given by the snapshot frequency.

Implement the function `simulate`. The function should for every time iteration:

- Apply the boundary condition to the ghost cells (T5).
- Do one time step forward (T4).
- Store the domain in a `.dat` file if the time iteration matches that of the snapshot frequency, `snapshot_freq`, by using the `domain_save` function.

After running your code, the produced `.mp4` file should resemble Fig. 7

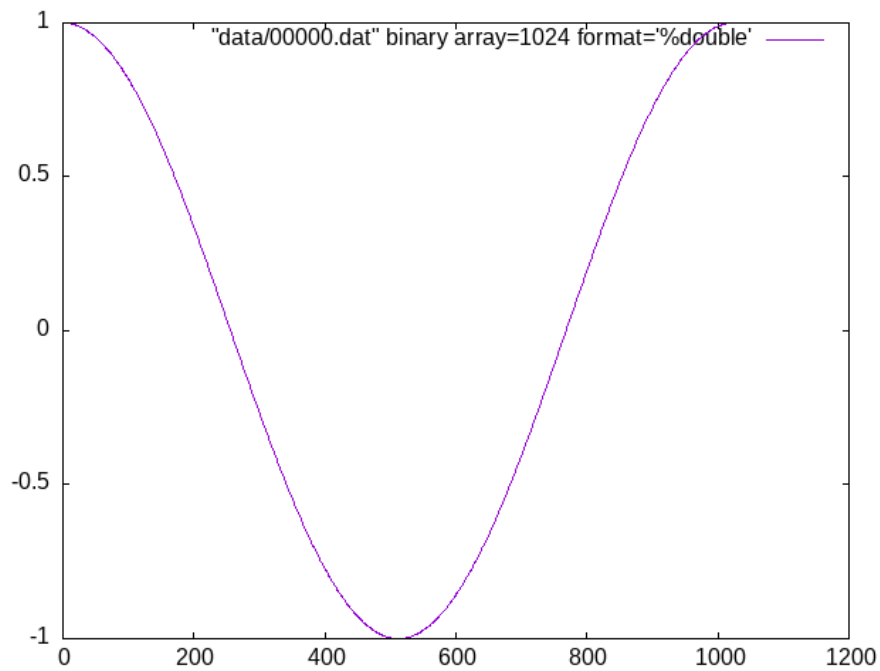


Figure 7: Image of the starting condition of the system you will write in the programming part.

where the wave moves up and down in a smooth manner.

2.2 Theory questions

1. Explain the pros and cons of using macros like

```

1  #define U_prv(i) buffers[0][(i)+1]
2  #define U(i)      buffers[1][(i)+1]
3  #define U_nxt(i) buffers[2][(i)+1]

```

for simulations like the wave equation.

2. Mention at least one other boundary condition that could have been applied instead of the Neumann (reflective) boundary condition. How will it affect the simulation?
3. What happens if you don't allocate memory in T1?
4. What is the difference between

```
1  float const *a;
```

and

```
1  float *const b;
```

?

3 Deliverables

Please deliver `.zip` file with the following contents containing your changes to the code:

- `wave_1d.c`
- `Makefile`
- A document with the answers to the theory questions.